Agentic Systems

Institute: Indian Institute of Information Technology Surat
Participants: ≥ Senior Undergrads
Lab: deepdynamicsai.github.io **Coordinator:** Dr. Pradeep Singh
Assignment 3

Problem. Design and implement, using basic Python and a small instruction-following large language model, an autonomous research agent that can answer questions by searching, reading, comparing, and verifying information distributed across multiple documents. The objective is to construct an *evidence-grounded agent* that decides which documents to inspect, maintains a record of the evidence obtained, resolves contradictory or outdated information, and produces a final answer supported by verifiable document references. Use an openly available model such as `Qwen/Qwen3-4B-Instruct-2507`, accessed through an API. Do not use agent frameworks such as LangChain, LangGraph, CrewAI, or AutoGen. The environment, document retrieval tools, memory, action validation, stopping logic, and evaluation must be implemented explicitly in Python.

Construct a synthetic document collection describing 12 fictional research projects, denoted by $p_1, \ldots, p_{12}$. For every project, generate exactly six documents: a registration document identifying its project leader and department; an initial budget authorization effective from February 1, 2026; a revised budget authorization effective from May 1, 2026; an initial approval-status notice effective from February 1, 2026; a revised approval-status notice effective from May 1, 2026; and a draft memorandum containing deliberately outdated or conflicting information. Thus, the complete collection contains 72 documents. Each document must have a unique identifier, title, issue date, effective date where applicable, status (`ISSUED` or `DRAFT`), and natural-language content. Assign fictional project names, leaders, departments, monetary amounts, and approval statuses using a reproducible random seed. Approval statuses must belong to {`APPROVED`, `PENDING`, `REJECTED`}. The underlying structured records used to generate the documents must remain hidden from the agent and must be used only for evaluation. The agent should have access exclusively to the documents through the permitted tools.

For each episode, the agent receives a question of the form: "As of date d, who is responsible for project p_i, which department does the project belong to, what is its authorized budget, and what is its approval status?" The query date d must be either March 1, 2026 or June 1, 2026. The answer cannot generally be obtained from a single document because the project leader, budget, and approval status occur in different documents. The agent must apply the following deterministic interpretation rules: only documents marked `ISSUED` are authoritative; a document becomes applicable on its effective date; for each attribute, the latest applicable issued document determines its value; and draft documents must never override issued documents. Consequently, the correct answer may change depending on the query date. The agent must discover the relevant evidence rather than receiving the underlying project records directly.

Provide the agent with three actions: `search(query,k)`, `read(doc_id)`, and `finish(answer)`. The `search` tool returns at most $k = 5$ document identifiers together with their titles, issue dates, and document statuses, ranked using a simple keyword-overlap or TF–IDF retrieval method implemented in Python. The `read` tool returns the complete content of a selected document, but only if its identifier has previously been returned by `search`. The `finish` action terminates the episode and submits the final answer. Assign a cost of 1 to each search, 2 to each document read, and 0 to the final submission. The total tool cost must not exceed $B = 12$, and the agent may make at most 15 LLM calls per episode. Repeated reading of the same document should be disallowed. The agent must therefore select its searches and document reads carefully rather than inspecting the entire collection.

At interaction step t, let M_t denote the agent's memory containing the original question, previous actions, returned search results, documents already read, and accumulated evidence. The LLM-based policy must select the next action according to $\pi_{\text{LLM}}(M_t, B_t)$, where B_t is the remaining tool budget. The LLM must return a strictly structured JSON response specifying the action, its arguments, and a short justification. For example, it may request `search` to locate budget notices, `read` to inspect a particular revision, or `finish` when sufficient evidence has been accumulated. The Python controller must parse the response, verify the action schema, check the remaining budget, reject unauthorized document access, execute valid actions, and append the resulting observations to memory. Invalid actions must not be executed, and their occurrences must be recorded. Each attempted LLM call counts toward the LLM-call limit. The model must never be allowed to access the complete document collection, inspect the hidden ground-truth records, or execute Python code supplied in its own responses.

The agent must maintain an explicit evidence record E_t, consisting of facts extracted from documents it

has actually read, together with their source identifiers, effective dates, and document statuses. For each requested attribute, the agent should identify a supporting document and determine whether that evidence is applicable to the query date. The final answer must contain the project leader, department, authorized budget in INR, approval status, and supporting document identifiers. A final claim is considered *grounded* only if it is supported by an issued document that has actually been retrieved and read, is applicable on date d, and has not been superseded by a later applicable issued document. At least three relevant sources should ordinarily be required: the project registration, the applicable budget authorization, and the applicable approval notice. The controller must verify the evidence identifiers and factual consistency of the submitted answer using the hidden structured records, but this verification must occur only after the agent has terminated. If the agent cannot establish the requested facts within its budget, it may explicitly return `INSUFFICIENT_EVIDENCE` rather than inventing an answer.

The resulting agent should follow the interaction cycle question $\rightarrow$ search $\rightarrow$ observation $\rightarrow$ read $\rightarrow$ evidence extraction $\rightarrow$ memory update $\rightarrow$ reasoning $\rightarrow$ next action or stop. Importantly, the next search or reading action must depend on information acquired through preceding actions. For example, after discovering that an initial budget notice exists, the agent may decide to search for an amendment; after finding contradictory statements, it must inspect their statuses and effective dates before determining which statement is authoritative. This distinction between predetermined retrieval and adaptive evidence acquisition is central to the assignment.

For comparison, implement two baselines. The first is a *fixed retrieval-augmented generation baseline* that performs one search using the original question, reads the first three returned documents, and asks the same LLM to generate a final answer without additional retrieval. The second is a deterministic keyword-based agent that selects searches and document reads using hand-written rules based on project identifiers, document types, and effective dates, without consulting an LLM. Both baselines must obey the same tool-cost limit, access restrictions, and evidence requirements as the proposed agent.

Evaluate all methods on the 24 questions obtained by combining the 12 projects with the two query dates. Report exact-answer accuracy, evidence-grounding accuracy, average tool cost, average number of searches, average number of documents read, and the fraction of episodes in which the available budget is exhausted. For LLM-based methods, also report the average number of LLM calls and the number of malformed or unauthorized action proposals. A possible episode loss is $J = 10\mathbf{1}\{\text{answer incorrect or evidence invalid}\} + 0.1C + 0.02N_{\text{LLM}}$, where C is the total tool cost and N_{LLM} is the number of model calls. An abstention should be reported separately and counted as an unsuccessful episode for the primary accuracy metric. Use the same generated corpus, questions, and retrieval implementation for all methods, and preserve complete action–observation traces for reproducibility.

Finally, analyze at least five unsuccessful episodes in detail and classify their failures into meaningful categories, such as retrieval failure, incorrect document selection, failure to recognize a revision, reliance on a draft document, insufficient evidence, hallucinated facts, premature stopping, or budget exhaustion. Explain whether these failures originate from the LLM policy, the available observations, memory management, retrieval limitations, or deterministic control logic. The purpose of the assignment is not simply to build a question-answering system, but to understand how an autonomous agent acquires information, constructs and verifies evidence, makes sequential decisions under resource constraints, and produces answers whose factual claims can be independently checked.